Chess met Heuristiek

Bas Terwijn

March 1, 2026

1 Introductie

Dit document behandelt de Chess game met heuristiek.

1.1 Doelen

Bekend raken met:

- Implementatie van het `Game_Abstract` interface voor Chess

2 Chess

We willen nu ook Chess (Nederlands: Schaken) implementeren als een game die het `Game_Abstract` interface volgt. Omdat Chess een stuk complexer is dan Tictactoe of Connect Four, maken we hier gebruik van de `chess` Python package die al veel van de complexe logica van Chess voor ons afhandelt. Voor de heursitiek maken we gebruik van `stockfish`, een open-source Chess engine die erg sterk is en de meeste kampioenschappen voor Chess Engines wint.

Installeer de `chess` package met:

```
$ pip install chess
```

Installeer de `stockfish` package voor Ubuntu met:

```
$ apt install stockfish
```

en voor Mac met:

```
$ brew install stockfish
```

Er zijn voor stockfish ook andere installatie methoden, zie Stockfish Download. Maar, pas dan indien nodig steeds de `STOCKFISH_PATH` variabele in de Python code aan zodat deze het pad aangeeft waar stockfish is geïnstalleerd.

De documentatie van deze software is hier te vinden:

- chess package
- Stockfish

Maar, omdat dit alles doorlezen teveel tijd kost geven we in file `chess_example.py` een voorbeeld-implementatie van een Chess game waarbij we ook de heuristiek al berekenen met gebruik van de stockfish engine. We doen dit met `engine.analyse(board, chess.engine.Limit(depth=0))` waarbij we niet vooruit denken(depth=0) en dus alleen de huidige spelsituatie laten beoordelen. Deze code werkt al, run deze met:

```
$ python chess_example.py
```

waarbij twee menselijke spelers tegen elkaar spelen. Als het bord wordt geprint wordt tegelijkertijd ook een duidelijkere SVG afbeelding van het bord gemaakt in file `chess.svg` die u kunt openen.

3 Opdracht `chess_adapter.py`

Het is nu uw taak om aan de hand van deze voorbeeld-code de `Chess_Adapter` class te implementeren in file `chess_adapter.py` zodat deze voldoet aan het `Game_Abstract` interface zodat onze Player classes er ook mee kunnen werken zoals weergegeven in figuur 1. We kunnen deze `Chess_Adapter` class zien als een vertaler/adapter tussen het interface van ons `Game_Abstract` interface van het interface van de `chess` package.

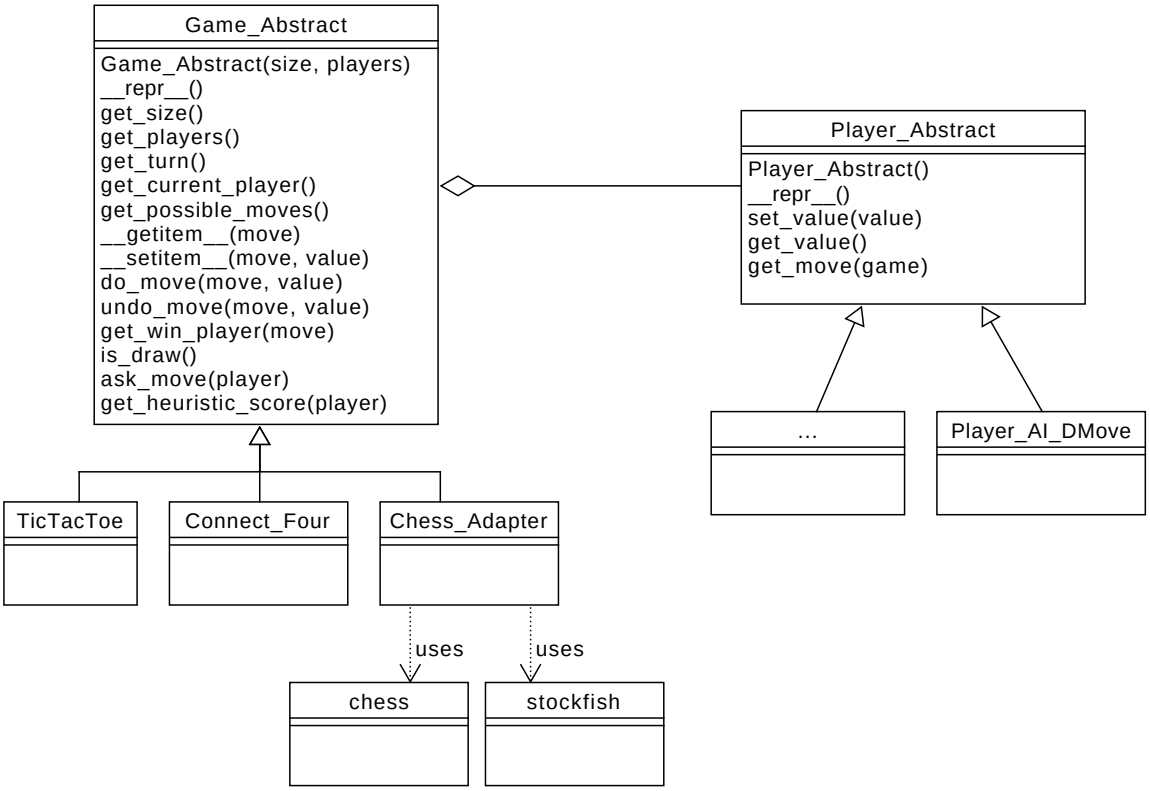

Figure 1: Chess als derde Game

De `Chess_Adapter` kan worden getest met:

```
$ pytest test_chess_adapter.py -vv
```

4 Alle Player classes

Als ook de `Chess_Adapter` goed werkt kunnen we nu alle Player classes, die we eerder hebben gemaakt, gebruiken om alle spellen te spelen doordat we voor spelers en spellen een duidelijk interface hebben gedefinieerd. Dit laat het nut zien van het gebruik van interfaces. Om alle Players die we eerder hebben gemaakt naar deze directory te kopieren kunnen we het volgende script gebruiken:

```
$ ./copy_old_players.sh
```

Waarna we met:

```
$ python play.py -g chess -p1 h -p2 d2h
```

een Chess spel kunnen spelen als mens tegen een `Player_AI_DMove` die twee zetten vooruit denkt en een heuristiek gebruikt. Of een ander spel spelen met andere type spelers, zie de opties van `play.py` met:

```
$ python play.py -h
```